BECE355L — Advanced Cloud Computing
Fall Semester 2026–27

Cloud-Native Parallel Computing Framework for Scalable Image Processing

Object Storage, Deep-Learning Operations, a Cost-Aware Planner and Work-Aware Autoscaling

Project Report

Python · OpenCV · ONNX Runtime · Redis · MinIO / Amazon S3 · FastAPI · Docker · Kubernetes · KEDA

Submitted by

Name	Register No.
Prapti Kurkuri	24BVD0185
Vaanya Garg	24BVD0139
Aditya Manoj	24BVD0070
Arrnav Trisal	24BVD0064

Submitted to: **Aparna Mohanty**

School of Electronics Engineering (SENSE), Vellore Institute of Technology, Vellore

31 October 2026

Cloud-Native Parallel Computing Framework for Scalable Image Processing

Object Storage, Deep-Learning Operations, a Cost-Aware Planner and Work-Aware Autoscaling

Prapti Kurkuri
Reg. No. 24BVD0185
SENSE
Vellore Institute of Technology
Vellore, Tamil Nadu, India
Praptikurkuri2024@vitstudent.ac.in

Vaanya Garg
Reg. No. 24BVD0139
SENSE
Vellore Institute of Technology
Vellore, Tamil Nadu, India
Vaanyagarg2024@vitstudent.ac.in

Aditya Manoj
Reg. No. 24BVD0070
SENSE
Vellore Institute of Technology
Vellore, Tamil Nadu, India
Adityamanoj2024@vitstudent.ac.in

Arrnav Trisal
Reg. No. 24BVD0064
SENSE
Vellore Institute of Technology
Vellore, Tamil Nadu, India
Arrnavtrisal2024@vitstudent.ac.in

Abstract*—Our earlier framework splits a large image into halo tiles, schedules one task per tile through a reliable Redis queue to stateless worker pods and stitches the results, with leases for fault tolerance and a queue-length autoscaler. Its evaluation exposed four weaknesses: every tile byte passed through Redis, which capped image size and concurrency; only cheap classical filters were supported, so most operations ran *slower* when distributed; the tile size and worker count were fixed by hand; and the autoscaler counted tiles, not work. This report presents four upgrades that address them. (1) Tiles, inputs and results move to S3-compatible object storage (MinIO locally, Amazon S3 in the cloud) and Redis carries only task IDs; Redis memory per 8192^2 job drops from 226 MB to 0.05 MB and accepted concurrent 8192^2 jobs rise from 2 to at least 12. (2) Two deep-learning operations, HED edge detection and Real-ESRGAN ×4 super-resolution, run under ONNX Runtime; their halos are calibrated empirically from the effective receptive field (80 px and 4 px instead of the theoretical 98 px and 34 px) and the stitcher is scale- and stride-aware. (3) A cost-aware planner predicts job time from a calibrated model of compute, per-tile overhead and a measured multi-worker capacity curve and chooses whether to distribute, the tile size and the worker count. Its prediction error is 25 % MAPE versus 40 % for a linear model, and it turns the 16× slowdown of naively tiled grayscale into a 1.11× result. (4) A work-aware autoscaler sizes the worker pool from queued compute-seconds and a latency target. On a replayed trace it met a 30 s latency target for every job with 25 % fewer worker pod-seconds than KEDA, while the CPU-based HPA met it for only 66 % of jobs. Neural operations reach only 2.5× (HED) and 2.4× (Real-ESRGAN) on a 16-thread laptop because they are memory-bandwidth bound, a result the planner models explicitly.

***Index Terms*—cloud-native computing, object storage, Kubernetes autoscaling, KEDA, performance model, tiled image processing, receptive field, ONNX Runtime, Redis, fault tolerance.**

I. Introduction

A. Background

Satellite scenes, whole-slide pathology images and microscopy mosaics reach hundreds of megapixels. Most image filters are *local*: each output pixel depends on a small neighbourhood. Such work can be split into tiles, processed in parallel and stitched, provided every tile carries a *halo* (overlap) wide enough that the stitched output equals whole-image processing. Our first version (v1) built this on Kubernetes: a FastAPI service partitions the image, Redis holds a reliable queue with one task per tile, stateless workers claim tiles under a lease and the last finisher reconstructs the output.

B. Problems Found in v1

The v1 evaluation [Sec. XI] showed where a simple design stops working:

1) **Redis was the data plane.** Every tile and result went through Redis, so an 8192^2 job used about 226 MB of Redis memory. With a 1 GB limit only 2 such jobs fit and 12288^2 images were rejected.
2) **Cheap operations got slower.** Canny ran at 0.45× and Gaussian blur at 0.41× when distributed, because moving a tile cost more than filtering it. Only the bilateral filter (4.2×) benefited.
3) **Configuration was manual.** The user picked the tile size; the system always distributed.
4) **The autoscaler counted tiles.** One HED tile costs about 1800 times a Canny tile of the same size, yet both counted as one unit of queue length.

C. Contributions

This report describes four upgrades, all implemented in the same repository, tested and measured on the same hardware:

- **Object-storage data plane** (S3/MinIO) with presigned result URLs, SSE-KMS, least-privilege IAM, lifecycle expiry and streaming tiled BigTIFF output for images too large for memory.
- **Deep-learning operations** (HED, Real-ESRGAN ×4) with empirical halo calibration, stride-aligned halos and scale-aware stitching.
- **A cost-aware adaptive planner** with a calibrated time model, a measured capacity curve per operation and online re-calibration from completed tasks.
- **A work-aware autoscaler** evaluated against the Kubernetes HPA and KEDA on an identical seeded trace.

II. Objectives

TABLE I
Project objectives and outcomes

#	Objective	Outcome
1	Remove the Redis memory ceiling	Achieved: S3/MinIO data plane; Redis holds IDs (Sec. V)
2	Support heavy neural operations seam-free	Achieved: HED and Real-ESRGAN, calibrated halos (Sec. VI)
3	Choose execution strategy automatically	Achieved: planner; 25 % MAPE (Sec. VII, X-C)
4	Scale on work instead of queue length	Achieved: work-aware scaler vs HPA, KEDA (Sec. VIII, X-D)
5	Keep v1 guarantees	Kept: halo-exact output, leases, retries; 43 tests pass

III. Related Work

Splitting and fault tolerance. MapReduce [6] established splitting a job into independent tasks with re-execution on failure; our tile tasks are map tasks with a spatial reduce. Leases [7] bound how long a crashed holder can block a resource; our tiles use leases renewed by heartbeats. **Image pipelines.** Halide [8] separates the algorithm from its schedule and treats tile size and recomputation of overlap as a trade-off; Distributed Halide [9] extends this to clusters and shows that communication, not compute, often limits scaling, which is exactly the effect our planner models. **Performance models.** Ernest [10] predicts the runtime of analytics jobs from a few small runs with a simple parametric model; our planner follows the same idea (calibrate, then extrapolate) with terms for per-tile overhead and contention. **Autoscaling.** The Kubernetes HPA [19] scales on resource utilisation and KEDA [18] on event-source metrics such as list length. Autopilot [11] at Google sets resources from observed usage rather than user limits. Bhosale *et al.* [1], [2] argue for elastic, workload-aware resizing of cloud HPC jobs, and Liu *et al.* [4] predict future workload requirements for containerised workflows; our work-aware scaler is a simple instance of both ideas. Chazapis *et al.* [5] and the fog pipeline of [3] support the premise that a thin scheduler on Kubernetes is a viable substrate for parallel compute. **Neural networks.** HED [12] and Real-ESRGAN [13] are fully convolutional, so they can be tiled, but their theoretical receptive fields are large. Luo *et al.* [14] showed the *effective* receptive field is much smaller and roughly Gaussian, which justifies calibrating the halo empirically.

IV. System Architecture

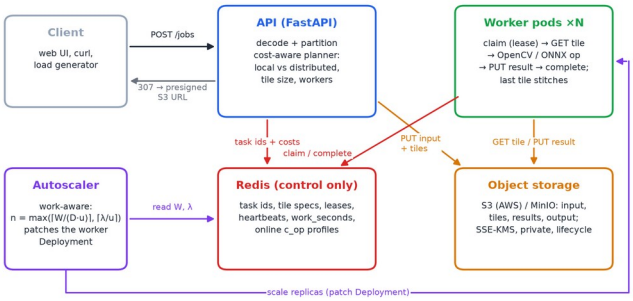


Fig. 1. Architecture after the upgrades. Bulk bytes (input, tiles, results) flow through object storage; Redis carries only task IDs, leases, per-tile costs and job state.

All components are built from one Docker image and run unchanged on a laptop, Docker Compose or Kubernetes:

- **API (FastAPI).** Accepts uploads, asks the planner for a plan, partitions the image, uploads input and tiles to the bucket, enqueues one task per tile with its predicted cost, and serves status and results (a 307 redirect to a presigned URL).
- **Redis.** Reliable queue (`pending`, `processing` lists and a lease sorted set), job hashes, per-tile cost hash, the queued *work-seconds* counter and the online profile. All state changes are Lua scripts so they are atomic.
- **Object store.** MinIO locally, Amazon S3 in the cloud. Keys are `jobs/<id>/input.<ext>`, `tiles/<i>.npy`, `results/<i>.npy`, `output.png|tif` and previews.
- **Workers.** Stateless; claim a tile, GET it, run the OpenCV or ONNX operation, PUT the result, complete it. The last finisher stitches (or streams a tiled TIFF).
- **Autoscaler.** Work-aware policy (ours), or the HPA / KEDA baselines; one is active at a time.

V. Upgrade 1: Object-Storage Data Plane

A. Design

v1 stored encoded tiles in Redis, so Redis memory, not CPU, limited the system. The upgrade applies the *claim-check* pattern: bulk bytes go to S3 and the queue entry carries only the tile ID. `imgframe/objstore.py` wraps boto3 with path-style addressing so the same code talks to MinIO and S3; tile uploads and downloads use a 32-thread pool. The API uploads all tiles *before* enqueuing any task, so a worker never claims a tile whose bytes are missing. Deleting a job removes its prefix and subtracts its remaining cost from the work counter. Redis mode is kept (`S3_BUCKET` unset) for tests and for the comparison below.

B. Result Delivery and Large Outputs

`GET /jobs/{id}/result` answers with a 307 redirect to a presigned URL valid for 300 s, so result bytes never pass through the API (a `proxy` mode exists for clients that cannot follow redirects). A separate signing client uses the public endpoint, because the in-cluster hostname of MinIO is not resolvable by a browser. Outputs above 150 MP (`STREAM_OUTPUT_MP`) are written tile by tile into a tiled BigTIFF with *tifffile*, so the aggregator never holds the full image; a 1200 px JPEG preview is written alongside.

C. Security

The AWS templates in `deploy/aws/` follow least privilege: the API role may put inputs and tiles, read results and presign; the worker role may only get tiles and put results. The bucket enforces Block Public Access and TLS-only access, encrypts with SSE-KMS [20], and expires `jobs/` after one day with a lifecycle rule. On EKS the roles are bound to Kubernetes service accounts with IRSA [21], so no keys are stored in pods. Locally, MinIO credentials live in a Kubernetes Secret. The AWS templates are written and syntax-checked but were not deployed (no AWS account was used).

VI. Upgrade 2: Deep-Learning Operations

A. Models

HED [12] produces a soft edge map from five VGG-16 side outputs. The public weights are a Caffe model and OpenCV 5 no longer imports Caffe, so we wrote `scripts/convert_hed.py`, which decodes the Caffe protobuf wire format directly and emits an equivalent ONNX graph (bilinear resize of side outputs, as in the PyTorch port). **Real-ESRGAN** [13] (`realesr-general-x4v3`, 1.2 M parameters) upscales ×4. Both run under ONNX Runtime [22] with one intra-op thread per process, so parallelism comes only from the framework and the comparison with sequential execution is fair. Model files are fetched by `scripts/fetch_models.sh` and verified by SHA-256.



Fig. 2. Canny vs HED on the same photograph, and the tiled-minus-whole error maps. With the calibrated halo the tiled HED output is visually identical to whole-image inference.

B. Halo Calibration from the Effective Receptive Field

The theoretical receptive-field radius of HED is 98 px and of Real-ESRGAN 34 px. Using it as the halo is exact but expensive: with 256 px tiles HED would compute 2.5× more pixels than needed. Following [14], `scripts/calibrate_halo.py` sweeps the halo, compares tiled with whole-image output and picks the smallest halo meeting two criteria: seam PSNR ≥ 40 dB (PSNR within 8 px of tile borders) and the 99.9th-percentile absolute error ≤ 8 grey levels. Table II and Fig. 3 show the sweep.

TABLE II
HALO CALIBRATION (256 PX TILES); MIDDLE ROW = CHOSEN HALO

Op	Halo (px)	Seam PSNR (dB)	p99.9 |err|	Extra pixels
HED	0	16.0	155	0 %
HED	80	42.3	8	164 %
HED	98	99.0	0	252 %
Real-ESRGAN	0	33.3	14	0 %
Real-ESRGAN	4	42.2	5	6 %
Real-ESRGAN	34	99.0	0	60 %

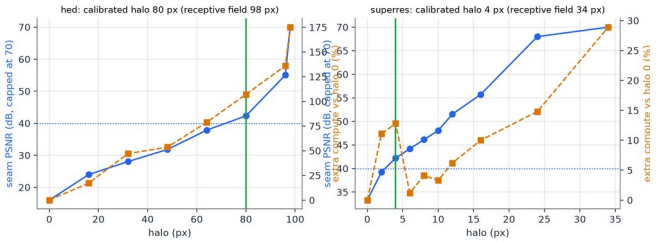


Fig. 3. Halo sweep: seam PSNR (left) and compute overhead (right). The chosen halo is the first point above both thresholds.

Two details mattered. First, HED downsamples by up to 16, so a tile whose origin is not a multiple of 16 sees a shifted pooling grid; `plan_tiles(..., align=16)` snaps *both* halo edges to the stride, which made the theoretical halo bit-exact (99 dB) where it had been worse than a 64 px halo. Second, Real-ESRGAN outputs 4× the input, so `crop_core` and `reconstruct` scale tile coordinates by the operation's `scale`. The calibrated halos cut extra computation from 252 % to 164 % (HED) and from 60 % to 6 % (Real-ESRGAN).

VII. UPGRADE 3: COST-AWARE ADAPTIVE PLANNER

A. Time Model

For an image of P pixels, operation cost c (s/px), K tiles of mean size $\bar{p}$ with halo h and n workers, the planner predicts

$$T_{\text{dist}} = T_{\text{split}} + T_{\text{comp}} + T_{\text{over}} + T_{\text{stitch}} \quad (1)$$

$$T_{\text{comp}} = \lceil K/n \rceil \cdot c \cdot \bar{p}_{\text{halo}} \cdot n \,/\, S(n) \quad (2)$$

$$o(p) = o_0 + o_{\text{px}} \cdot p_{\text{halo}} \cdot (1 + s2) \quad (3)$$

where $S(n)$ is the measured *capacity curve* of the operation (throughput of n concurrently busy workers relative to one), $o(p)$ the per-tile overhead (encode, transfer, queue round trip) and s the output scale. Overheads are paid in $\lceil K/n \rceil$ waves on the cluster; on the single-process engine, where dispatch is serial, $T = T_{\text{serial}} + \max(T_{\text{comp}} + T_{\text{over}}, K \cdot o)$. The local alternative is $T_{\text{local}} = c \cdot P$ (plus split and stitch when it runs as a single tile on the cluster). The planner evaluates tile sizes $\{256, 512, 1024, 2048\}$ and every worker count up to the available workers, adds a 20 % safety margin to the distributed estimate, and among plans within 2 % of the best picks the one with fewer workers.

B. Calibration

`scripts/profile_ops.py` measures c for each operation on four tile sizes, the capacity curve for 1–16 workers and the per-tile overhead of each backend (in-process engine, cluster with S3, cluster with Redis), and writes `imgframe/profiles.json` (Table III, Fig. 4). Workers keep refining c online: after each tile they add an exponential moving average ($\alpha = 0.05$) to Redis, normalised for the contention at that moment as $c \leftarrow t \cdot S(b)/(b \cdot p)$, where b is the number of busy workers.

TABLE III
MEASURED PER-PIXEL COST AND CAPACITY CURVE (16-THREAD HOST)

Op	c (ns/px)	S(4)	S(8)	S(16)
Grayscale	0.3	1.06	2.55	3.80
Canny	2.6	2.24	4.29	5.97
Gaussian	4.2	3.39	4.67	6.32
Bilateral	73.9	3.79	6.46	7.29
HED	4,545	3.37	5.07	4.54
Real-ESRGAN	23,814	2.16	1.96	1.88

Per-tile overhead on the cluster is o_0 = 2.59 ms + 3.85 ns/px with S3 and 3.29 ns/px with Redis; a 512^2 tile PUT+GET takes 5.7 ms on MinIO and 0.52 ms on Redis. Since Canny costs only 2.6 ns/px, a 512^2 Canny tile computes in 0.7 ms but costs ≈ 5 ms to move, which is why cheap operations must not be distributed finely.

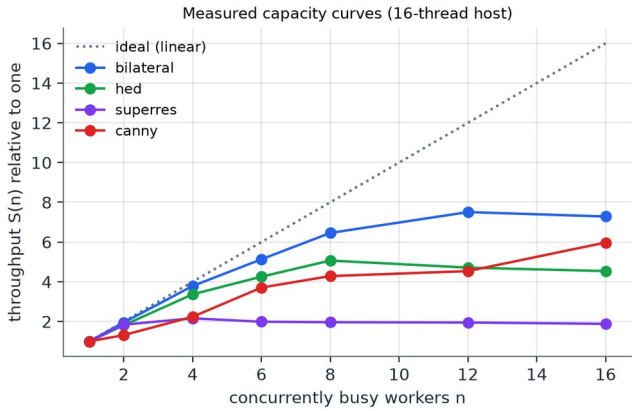


Fig. 4. Capacity curves S(n). Bilateral scales to ≈ 7× on 16 threads; HED peaks at ≈ 5× at 8 workers and Real-ESRGAN saturates at ≈ 2× from 4 workers: both are memory-bandwidth bound on this host.

VIII. Upgrade 4: Work-Aware Autoscaling

When the API enqueues a tile it stores the tile's predicted compute time and adds it to a Redis counter W (queued work-seconds). The completion, give-up and delete Lua scripts subtract exactly the stored cost, so W never drifts. Every 3 s the autoscaler reads W, estimates the arrival rate of work λ (s of work per s) as an EMA of the increase of a monotonic *work-added* counter, and sets

$$n = \text{clamp}(\max(\lceil W / (D \cdot u) \rceil, \lceil \lambda / u \rceil), n_{\min}, n_{\max}) \qquad (4)$$

with $D = L - t_{\text{start}}$ the time left to drain the queue once a pod has started (latency target L = 30 s, pod start-up t_{start} = 5 s) and u = 0.8 the target utilisation. The first term drains the backlog within the latency target; the second keeps up with the steady arrival rate. Scale-ups are applied at once. A reduction is applied after a 30 s stabilisation window, as in the baselines, with one exception: when $W < 0.05$ s (nothing queued or in progress) for 10 s (`IDLE_SCALE_DOWN_S`), the pool drops straight to $n_{\min}$. An empty queue is exact information, unlike the slowly decaying EMA λ, and a new burst is answered by an immediate scale-up whose start-up time is already budgeted in D. Unlike a queue-length rule, ten HED tiles and ten Canny tiles produce very different W, so the pool reacts to how much work is waiting.

Baselines. *HPA*: CPU utilisation target 70 % of the worker request (`autoscaling/v2`, 30 s scale-down window) [19]. *KEDA* 2.21 [18]: Redis Lists scaler on `queue:pending`, 16 tiles per replica, polling every 3 s. All three use 1–8 replicas; `scripts/k8s_scaler.sh` switches between them so exactly one controls the worker Deployment.

IX. Experimental Setup

All experiments ran on one laptop: Intel Core i7-11800H (8 cores, 16 threads), 22 GB RAM, Ubuntu 25.10, Python 3.13, OpenCV 5.0, ONNX Runtime 1.31, Redis 7.2, MinIO (Chainguard image). Kubernetes experiments use Minikube 1.37 (Docker driver, 8 CPUs, 12 GB) with metrics-server and KEDA 2.21. Engine benchmarks use 8192^2 synthetic images for classical operations, a 2048^2 photograph for HED and a 1024^2 one for Real-ESRGAN. Each point is the median of three runs unless stated. Because workers share one host, results show *relative* behaviour; absolute speedups on a multi-node cluster would differ.

X. Results

A. Object Storage vs Redis Data Plane

TABLE IV
Storage Study

Metric (Redis maxmemory 1 GB)	Redis mode	S3 mode
Largest image accepted	8192^2 (12288^2 → 503)	≥ 16384^2
Concurrent 8192^2 jobs accepted	2	12 (all tried)
Redis memory per 8192^2 job	226.4 MB	0.054 MB
I/O per 512^2 tile	7.2 ms	8.6 ms
Bilateral 4096^2, end-to-end	0.82 s	1.09 s

Moving bytes out of Redis reduces its footprint per job by a factor of 4,215 and removes the concurrency limit (the 12 jobs we tried were all accepted; the remaining limit is the configurable tile-count back-pressure, 20 000 queued tiles). The price is latency: each tile costs 1.3 ms more I/O and a small bilateral job takes 33 % longer, mostly from uploading the input. For large or many concurrent jobs this trade is clearly worth it; the planner already accounts for the higher per-tile overhead.

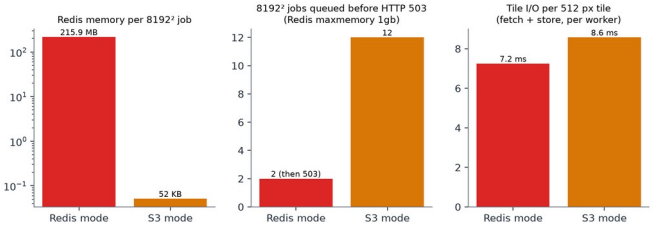


Fig. 5. Redis memory per 8192^2 job (log scale) and 8192^2 jobs accepted before HTTP 503, per storage mode.

B. Parallel Scaling of Neural Operations

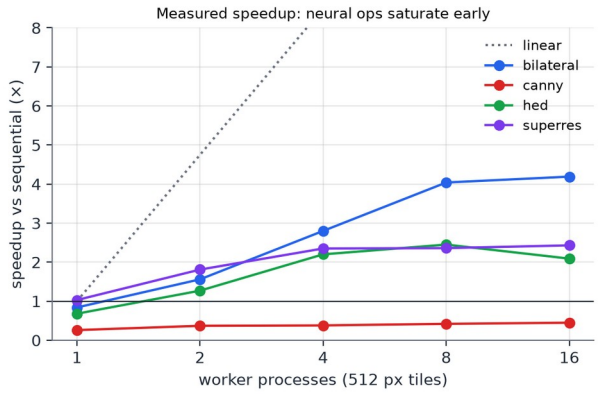


Fig. 6. Speedup vs workers. Bilateral (compute-bound) scales furthest; HED peaks at 8 workers and Real-ESRGAN plateaus from 4.

HED reaches 2.45× at 8 workers and Real-ESRGAN 2.43× at 16. The upgrade plan expected near-linear speedup for neural operations because they are compute-heavy, but on one host they are limited by memory bandwidth: convolution activations of many concurrent tiles exceed the caches, so extra workers wait on DRAM. HED's per-process memory also grows with workers (peak RSS ≈ 3 GB per 8 workers). On a multi-node cluster, where every node brings its own memory bandwidth, the curve would continue; the planner learns the measured curve, so it stops adding workers where they no longer help.

C. Planner

TABLE V
ENGINE: NAIVE (512 PX, 16 WORKERS) VS PLANNER; SPEEDUP OVER SEQUENTIAL

Op (image)	Seq. (s)	Naive	Planner	Plan
Grayscale (8192²)	0.03	0.06×	1.11×	local
Canny (8192²)	0.18	0.55×	0.98×	local
Gaussian (8192²)	0.28	0.54×	1.00×	local
Sobel (8192²)	0.19	0.60×	1.02×	local
Bilateral (8192²)	4.96	5.08×	4.94×	n=12, 512 px
HED (2048²)	20.31	2.00×	2.50×	n=8, 512 px
Real-ESRGAN (1024²)	25.20	2.16×	2.23×	n=4, 512 px

Naive tiling makes every cheap operation slower (grayscale runs at 0.06×). The planner recognises that and runs them locally, recovering ≈ 1×. For bilateral it chooses 12 workers, within 3 % of naive 16-worker tiling, and for the neural operations it picks fewer workers than naive (8 for HED, 4 for Real-ESRGAN) and is faster (2.50× vs 2.00× for HED) because it avoids the contention region of the capacity curve.

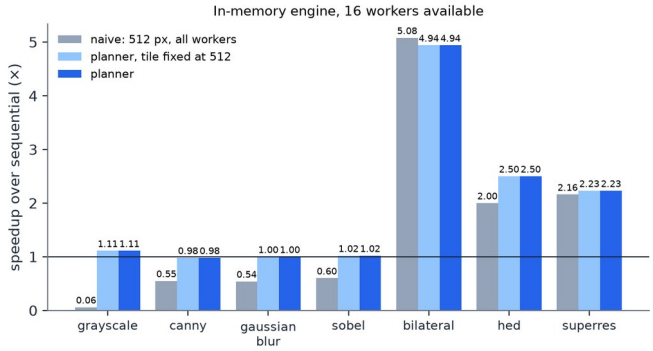


Fig. 7. Speedup over sequential on the in-memory engine: naive 512 px tiling on all workers vs the planner's choice.

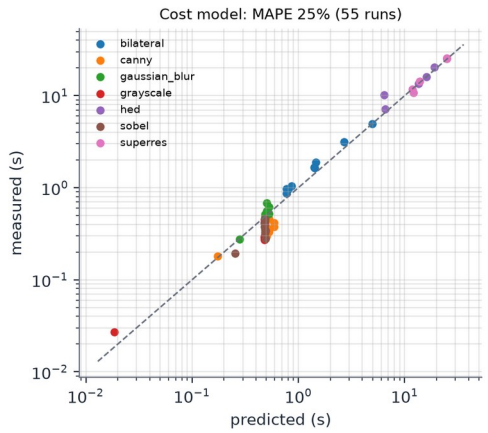


Fig. 8. Predicted vs measured time for all benchmarked configurations, refined model (capacity curve, serial dispatch) vs simple linear model.

Across 55 engine configurations the refined model has 25.0 % mean absolute percentage error against 40.2 % for the simple model ($T = c{\cdot}P/n + K{\cdot}o$). The gain is largest for the operations that matter most: HED 10 % vs 27 %, Real-ESRGAN 4 % vs 20 %, bilateral 15 % vs 28 %. Errors remain high (≈ 40 %) for sub-0.2 s jobs, where timing noise dominates; there a wrong choice costs milliseconds.

TABLE VI
FULL FRAMEWORK WITH S3 (8 WORKERS), END-TO-END JOB TIME

Op	Naive (s)	Planner (s)	Planner plan
Canny	0.70	0.61	16 × 1024 px
Gaussian	0.87	0.84	16 × 1024 px
Sobel	0.74	0.64	16 × 1024 px
Bilateral	1.05	0.99	16 × 1024 px
HED	8.55	8.43	16 × 512 px
Real-ESRGAN	9.63	10.75	4 × 512 px

Through the full framework (HTTP upload, S3, Redis, worker processes) the planner replaces 64 tiles of 512 px with 16 tiles of 1024 px for classical operations, cutting end-to-end time by 4–14 %; for the neural operations it agrees with the naive choice (differences within noise). The cluster model's MAPE is 21.7 %. One honest observation: in-memory Canny on 4096² takes 48 ms, so for cheap operations the whole job is dominated by upload and storage, and the best plan would be to run them in the API process without queuing. We list that under future work.

D. Autoscaling: HPA vs KEDA vs Work-Aware

The trace (`scripts/trace_replay.py`, seed 7) has 32 jobs in four phases: two bursts of cheap jobs (grayscale, Canny on 4096²) and two bursts of expensive jobs (bilateral 4096², HED 2048²) with exponential inter-arrival times and 15 s pauses between phases. Each scaler replayed it 3 times, interleaved, starting from one idle worker. Table VII reports mean ± standard deviation; the latency target is 30 s.

TABLE VII
AUTOSCALER COMPARISON ON MINIKUBE (MEAN ± S.D.)

Metric	HPA	KEDA	Work-aware
p50 latency (s)	25.7 ± 3.0	2.2 ± 0.3	2.7 ± 0.3
p95 latency (s)	42.7 ± 3.9	14.8 ± 5.9	12.5 ± 0.2
Jobs within target (%)	65.6 ± 15.6	100.0 ± 0.0	100.0 ± 0.0
Worker pod-seconds	328 ± 11	657 ± 28	490 ± 12
First scale-up (s)	78.5 ± 15.4	3.4 ± 0.6	33.1 ± 0.6
Scaling events	1.7 ± 0.6	6.0 ± 1.0	6.0 ± 0.0

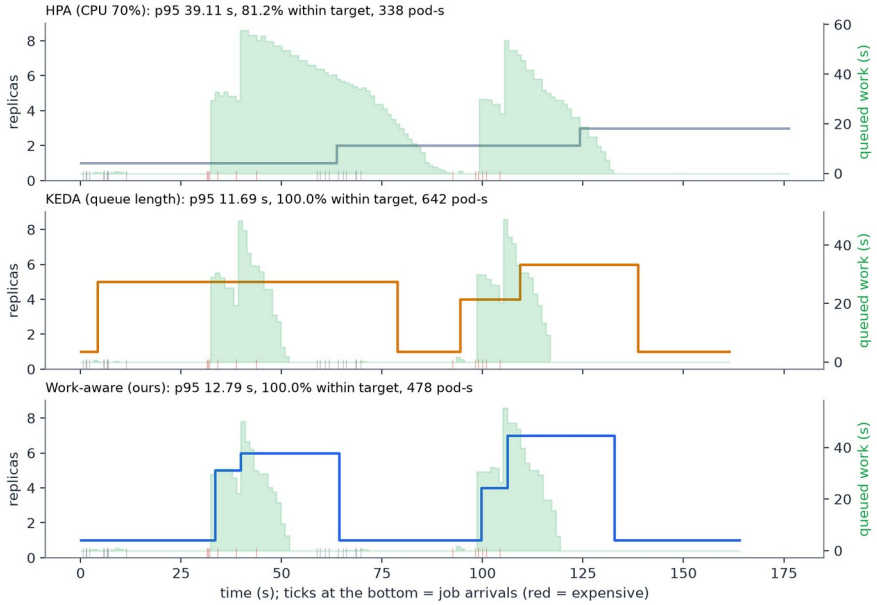


Fig. 9. Replica count (line) and queued work-seconds (shaded) during one run of each scaler; ticks mark job arrivals (red = expensive).

HPA is the cheapest (328 pod-s) but misses the target: only 66 % of jobs finish within 30 s and p95 latency is 43 s. CPU utilisation is averaged over a metrics window and the worker request is large, so its first scale-up came 78 s after the first arrival, by which time the expensive burst was mostly over.

KEDA and the **work-aware** scaler both met the target for every job, with similar p95 latency (14.8 ± 5.9 s vs 12.5 ± 0.2 s); the work-aware scaler used 25 % fewer worker pod-seconds (490 vs 657). The replica traces show why: KEDA scales up within 4 s of the *cheap* bursts, because 10 grayscale/Canny jobs queue hundreds of tiles, although those jobs finish in about two seconds on one worker. The work-aware scaler sees only a few seconds of queued work there and stays at one replica; its first scale-up (33 s) is the start of the expensive burst, which it meets with 5–7 workers.

Idle scale-down. Our first version kept its peak replica count for about 20 s after the queue drained: the arrival-rate EMA λ decays slowly and every reduction waited the full 30 s window. Adding the idle rule (Sec. VIII) cut worker pod-seconds from 560 ± 37 to 490 ± 12 (12 % less), while p95 latency only moved from 12.3 s to 12.5 s, with 100 % of jobs still within the target (3 runs each, same trace). With one trace on one host the differences between KEDA and the work-aware scaler should be read as indicative; the robust result is that scaling on work avoids reacting to cheap tile counts while still meeting the latency target.

E. Correctness and Testing

The test suite has 43 tests (all pass): tiling and exact reconstruction for every classical operation, the reliable queue (claim, duplicate completion, lease expiry, retries), end-to-end HTTP runs including a crashed worker, and 17 new tests for the upgrades: S3-mode jobs keep pixels out of Redis, proxy and redirect delivery, queue-depth back-pressure, crash recovery in S3 mode, stride-aligned halos, scale-aware stitching, streamed TIFF equal to in-memory output, HED tiled vs whole (PSNR), Real-ESRGAN end to end, planner decisions, the online cost EMA, exact work-seconds accounting with retries and duplicates, the work-aware replica rule and the idle scale-down. Fault tolerance from v1 still holds: a worker killed mid-job on Kubernetes was replaced and its leased tiles were re-run without loss.

XI. Baseline Behaviour of v1

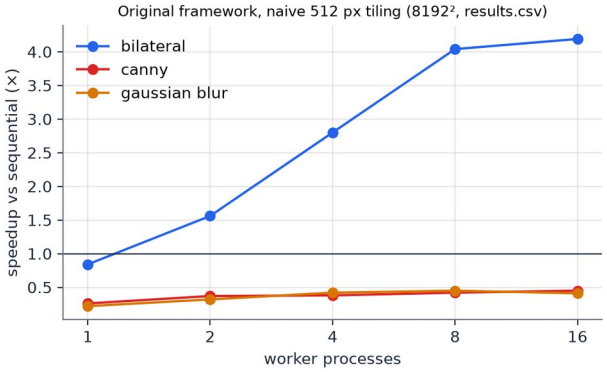


Fig. 10. v1 measurements that motivated the upgrades: only the CPU-heavy bilateral filter benefited from distribution.

For reference, v1 on an 8192² image reached 4.04× (8 workers) and 4.19× (16) for bilateral on the parallel engine, and 1.40× end to end through the framework, bounded by serial upload and aggregation (Amdahl's law [15]); Canny [16] and Gaussian blur were slower when distributed. The upgrades keep these guarantees and remove the four limits listed in Sec. I-B.

XII. Deployment and Mapping to AWS

`k8s/` contains the namespace, ConfigMap and Secret, Redis (256 MB, no persistence: it holds control data only), MinIO (NodePort 30900 for presigned downloads), the API (NodePort 30080), the worker Deployment (2 GiB limit for the neural models) and the autoscaler with RBAC limited to scaling the worker Deployment. `k8s/optional/` holds the HPA and KEDA baselines. Docker Compose runs the same stack locally.

TABLE VIII
Local deployment and AWS equivalents

Component	Minikube	AWS
Object store	MinIO	Amazon S3 (SSE-KMS, lifecycle)

Queue / state	Redis Deployment	ElastiCache for Redis
Orchestration	Minikube	Amazon EKS
Pod scaling	Work-aware / HPA / KEDA	Same, + Karpenter for nodes
Credentials	Kubernetes Secret	IRSA roles per service account
Ingress	NodePort	ALB (Load Balancer Controller)
Images	Minikube Docker	Amazon ECR

XIII. Limitations and Future Work

- **Single host.** All workers share one CPU and memory bus, which caps neural speedup at ≈ 2–2.5×. A multi-node EKS run would test the capacity model on real network and storage latencies.
- **AWS not exercised.** The S3, IAM, KMS and IRSA templates are complete but were not deployed; MinIO stood in for S3.
- **Cheap operations on the cluster.** They should run in-process at the API (or on a GPU) instead of through the queue; the planner already predicts when this wins.
- **Model error.** 20–25 % MAPE is useful for decisions but not for SLAs; per-host calibration and a memory-bandwidth term would help.
- **Not implemented from the plan's optional extras:** GPU workers, spot/pre-emptible node pools, Cloud-Optimised GeoTIFF range reads with a deep-zoom viewer, and learned (not analytical) cost models.

XIV. Conclusion

The upgrades turn a working tile-parallel framework into one that knows the cost of what it runs. Moving bytes to object storage removed the memory ceiling at the cost of a few milliseconds per tile; calibrated halos made deep-learning operations seam-free at a fraction of the theoretical overhead; a calibrated model lets the planner avoid the slowdowns of naive distribution and the contention region of memory-bound models; and scaling on queued work instead of queue length lets the pool react to what the queue actually costs. The main lesson is that on a cloud platform the decision *whether* and *how finely* to distribute matters as much as the parallel machinery itself.

References

[1] A. Bhosale, A. Tahilyani, L. Kale, and S. Kokkila-Schumacher, "Towards an adaptive runtime system for cloud-native HPC," arXiv preprint arXiv:2603.14630, Mar. 2026.

[2] A. Bhosale, K. Chandrasekar, L. Kale, and S. Kokkila-Schumacher, "An elastic job scheduler for HPC applications on the cloud," in *Proc. SC Workshops '25*, St. Louis, MO, USA, Nov. 2025, doi: 10.1145/3731599.3767358.

[3] "Serverless data pipeline architecture supporting distributed machine learning on fog devices," *Comput. Commun.*, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0140366426000952

[4] D. Liu, Y. Xia, C. Shan, K. Tian, and Y. Zhan, "A Kubernetes-based scheme for efficient resource allocation in containerized workflow," *Future Gener. Comput. Syst.*, vol. 166, May 2025, Art. no. 107699.

[5] A. Chazapis, E. Maliaroudakis, F. Nikolaidis, M. Marazakis, and A. Bilas, "Running cloud-native workloads on HPC with high-performance Kubernetes," arXiv preprint arXiv:2409.16919, Sep. 2024.

[6] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. 6th USENIX OSDI*, 2004, pp. 137–150.

[7] C. Gray and D. Cheriton, "Leases: An efficient fault-tolerant mechanism for distributed file cache consistency," in *Proc. 12th ACM SOSP*, 1989, pp. 202–210.

[8] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," in *Proc. ACM PLDI*, 2013, pp. 519–530.

[9] T. Denniston, S. Kamil, and S. Amarasinghe, "Distributed Halide," in *Proc. 21st ACM PPoPP*, 2016, Art. no. 5.

[10] S. Venkataraman, Z. Yang, M. Franklin, B. Recht, and I. Stoica, "Ernest: Efficient performance prediction for large-scale advanced analytics," in *Proc. 13th USENIX NSDI*, 2016, pp. 363–378.

[11] K. Rzadca *et al.*, "Autopilot: Workload autoscaling at Google," in *Proc. 15th EuroSys*, 2020, Art. no. 16.

[12] S. Xie and Z. Tu, "Holistically-nested edge detection," in *Proc. IEEE ICCV*, 2015, pp. 1395–1403.

[13] X. Wang, L. Xie, C. Dong, and Y. Shan, "Real-ESRGAN: Training real-world blind super-resolution with pure synthetic data," in *Proc. IEEE/CVF ICCV Workshops*, 2021, pp. 1905–1914.

[14] W. Luo, Y. Li, R. Urtasun, and R. Zemel, "Understanding the effective receptive field in deep convolutional neural networks," in *Proc. NeurIPS*, 2016, pp. 4898–4906.

[15] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Comput. Conf.*, vol. 30, 1967, pp. 483–485.

[16] J. Canny, "A computational approach to edge detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-8, no. 6, pp. 679–698, Nov. 1986.

[17] C. Tomasi and R. Manduchi, "Bilateral filtering for gray and color images," in *Proc. 6th ICCV*, 1998, pp. 839–846.

[18] KEDA Authors, "KEDA: Kubernetes event-driven autoscaling, v2.21 documentation — Redis Lists scaler." [Online]. Available: https://keda.sh/docs/ (accessed Oct. 2026).

[19] The Kubernetes Authors, "Horizontal Pod Autoscaling." [Online]. Available: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/ (accessed Oct. 2026).

[20] Amazon Web Services, "Amazon S3 user guide: presigned URLs, server-side encryption with AWS KMS, Block Public Access, lifecycle configuration." [Online]. Available: https://docs.aws.amazon.com/s3/ (accessed Oct. 2026).

[21] Amazon Web Services, "IAM roles for service accounts (Amazon EKS user guide)." [Online]. Available: https://docs.aws.amazon.com/eks/ (accessed Oct. 2026).

[22] ONNX Runtime developers, "ONNX Runtime: threading and performance tuning." [Online]. Available: https://onnxruntime.ai/docs/ (accessed Oct. 2026).

[23] Redis Ltd., "Redis documentation: Lua scripting, lists and the reliable-queue pattern." [Online]. Available: https://redis.io/docs/ (accessed Oct. 2026).

[24] MinIO, Inc., "MinIO object storage: S3-compatible API." [Online]. Available: https://min.io/docs/ (accessed Oct. 2026).

Appendix A: Project Structure

```
cloud proj/
├── imgframe/            core package
│   ├── config.py        settings from environment variables
│   ├── ops.py           OpenCV + ONNX operations, halo, scale, stride
│   ├── models.py        ONNX Runtime session cache
│   ├── tiling.py        stride-aligned plan, split, scale-aware reconstruct, tiled TIFF
│   ├── store.py         Redis data model, reliable queue + work counter (Lua)
│   ├── objstore.py      S3 / MinIO client (boto3), presigning
│   ├── planner.py       cost model + plan search
│   ├── profiles.json    calibrated costs, overheads, capacity curves
│   └── calibration.json calibrated halos
├── api/                 FastAPI service + web UI
├── worker/              worker process (+ online profile updates)
├── autoscaler/          work-aware (and queue-length) autoscaler
├── benchmark/           bench_local, bench_distributed, bench_planner, bench_storage
├── k8s/                 manifests; optional/ HPA and KEDA baselines
├── deploy/aws/          S3 bucket policy, IAM, lifecycle, IRSA (templates)
├── scripts/             run_local, convert_hed, calibrate_halo, profile_ops,
│                        trace_replay, k8s_scaler, fetch_models, install_k8s_addons
├── tests/               pytest suite
└── docs/                report, slides, guide, figures
```

Appendix B: New Configuration

Variable	Default	Meaning
S3_BUCKET	(unset)	Enables object-storage mode
S3_ENDPOINT / S3_PUBLIC_ENDPOINT	—	Internal endpoint / endpoint used in presigned URLs
S3_SSE, S3_KMS_KEY_ID	—	Server-side encryption (aws:kms on AWS)
PRESIGN_SECONDS	300	Validity of result URLs
RESULT_DELIVERY	redirect	redirect (307 to presigned URL) or proxy
MAX_QUEUED_TILES	20000	Back-pressure limit in S3 mode
STREAM_OUTPUT_MP	150	Outputs above this are written as tiled BigTIFF
PLANNER, PLANNER_MARGIN	1, 0.2	Enable planner; safety margin on distributed estimate
PROFILE_EMA	0.05	Weight of online cost updates
POLICY	work	Autoscaler policy: work or queue
LATENCY_TARGET_S, TARGET_UTILIZATION, POD_STARTUP_S	30, 0.8, 5	Work-aware scaler parameters
SCALE_DOWN_DELAY, IDLE_SCALE_DOWN_S	30, 10	Delay before a reduction; delay before an idle pool drops to MIN